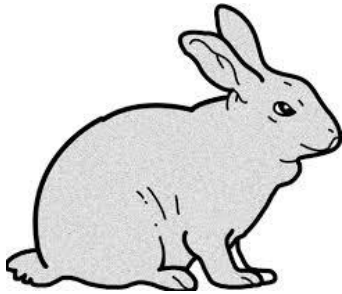
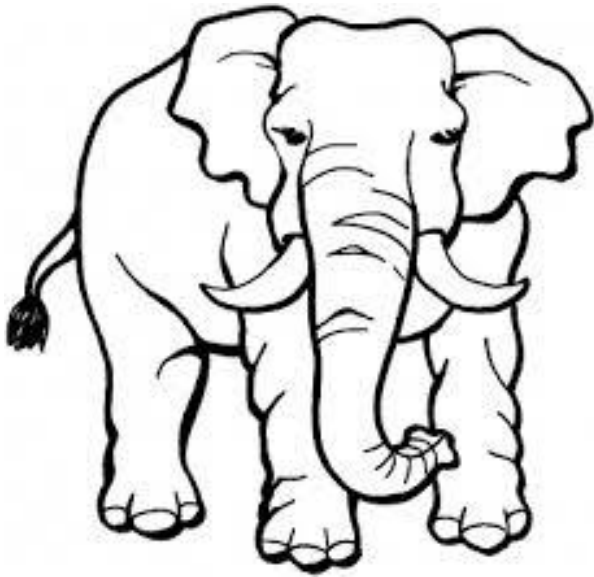


Memory Hierarchy

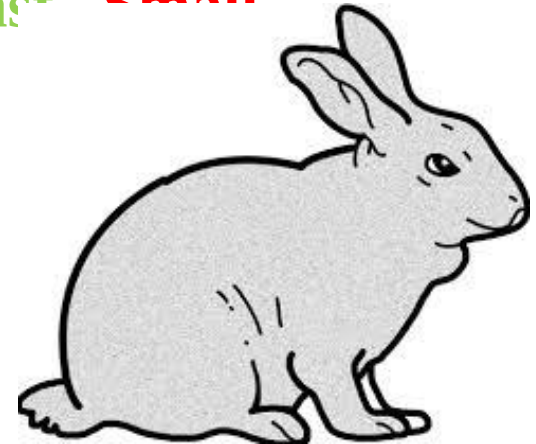
Actual Memory Systems



+



Type1: Fast Small



Can we achieve?: Fast, Large

Type2: Slow, Large

(Cache Principle)

Memory System Performance Example

```
void copyij(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (i = 0; i < 2048; i++)
        for (j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}
```

59,393,288 clock cycles

```
void copyji(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (j = 0; j < 2048; j++)
        for (i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}
```

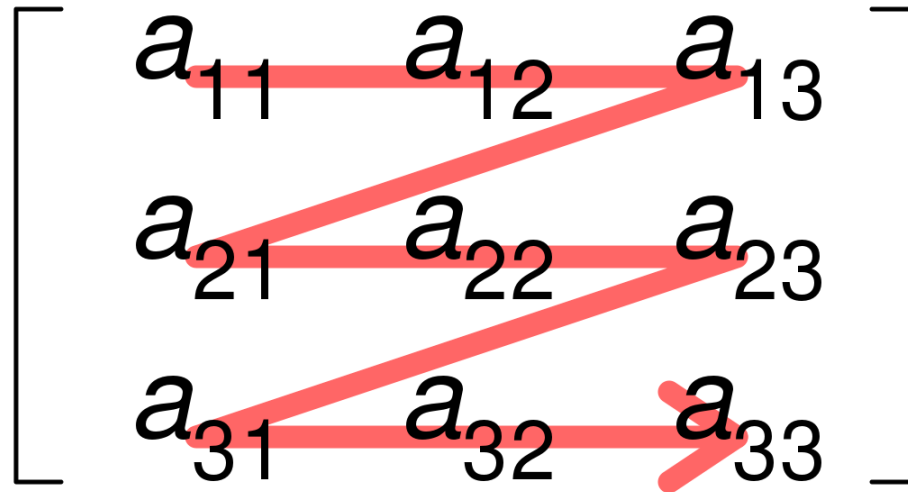
1,277,877,876 clock cycles

21.5 times slower!

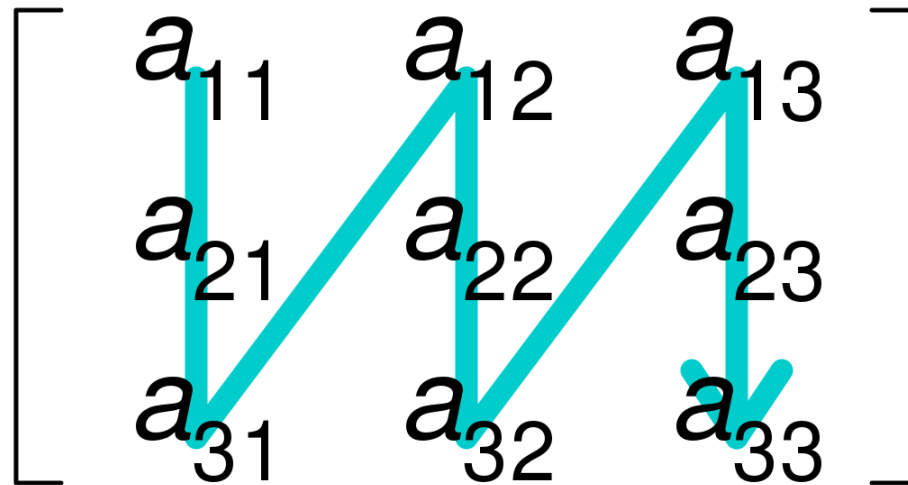
(Measured on 2GHz
Intel Pentium 4)

- Hierarchical memory organization
- Performance depends on access patterns

Row-major order



Column-major order

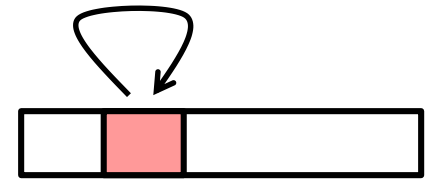


Locality

- **Principle of Locality:** Programs tend to use data and instructions with addresses near or equal to those they have used recently

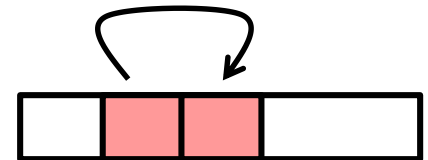
- **Temporal locality:**

- Recently referenced items are likely to be referenced again in the near future



- **Spatial locality:**

- Items with nearby addresses tend to be referenced close together in time



Locality Example

- Data references
 - Reference array elements in succession
 - Reference variable `sum` each iteration.
- Instruction references
 - Reference instructions in sequence.
 - Cycle through loop repeatedly.

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += a[i];  
return sum;
```

Spatial locality

Temporal locality

Spatial locality

Temporal locality

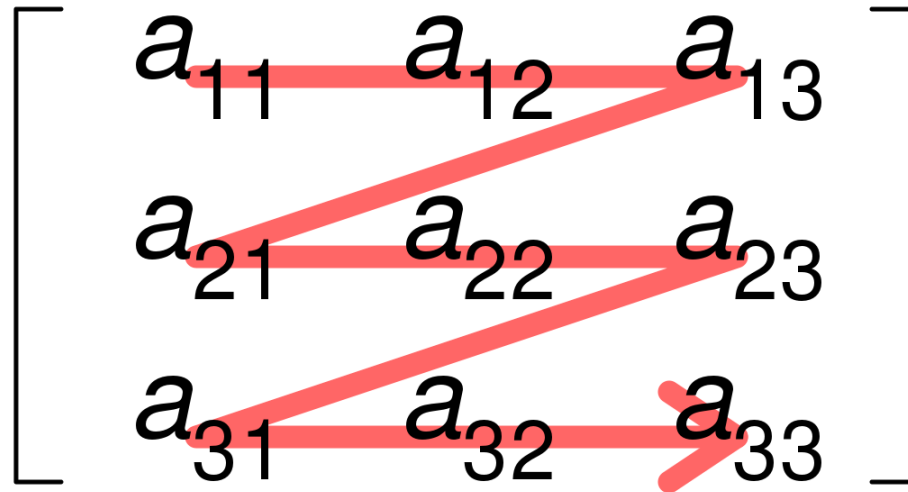
Qualitative Estimates of Locality

- **Claim:** Being able to look at code and get a qualitative sense of its locality is a key skill for a professional programmer.
- **Question:** Does this function have good locality with respect to array *a*?

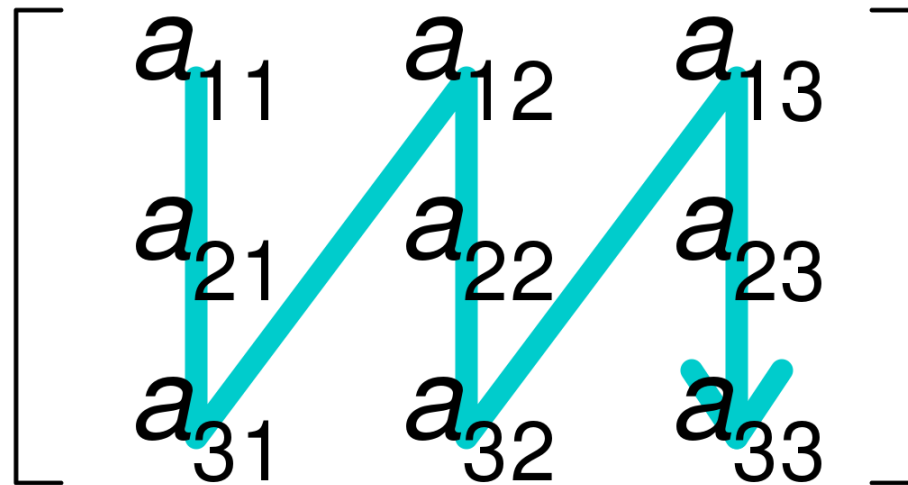
```
int sum_array_rows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Row-major order



Column-major order



1	2	3
4	5	6
7	8	9



row-major

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

1	2	3
4	5	6
7	8	9



column-major

1	4	7	2	5	8	3	6	9
---	---	---	---	---	---	---	---	---

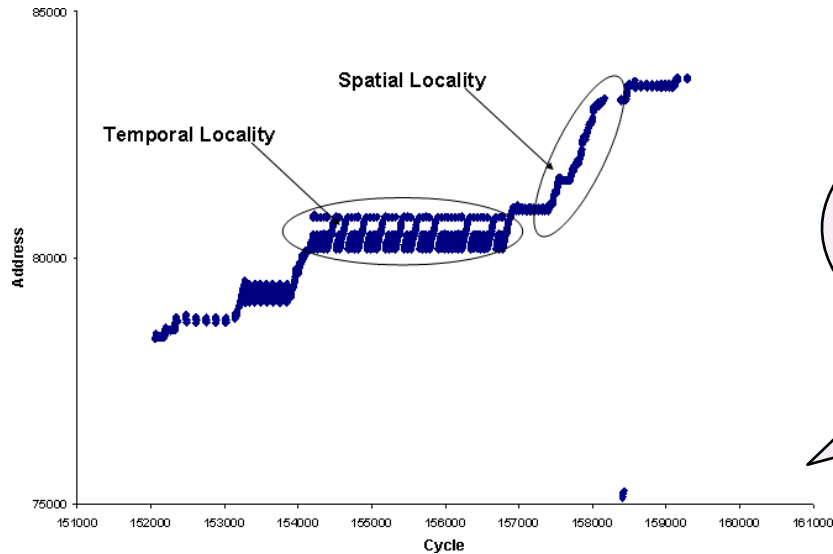
Locality Example

- **Question:** Does this function have good locality with respect to array *a*?

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Locality reference example

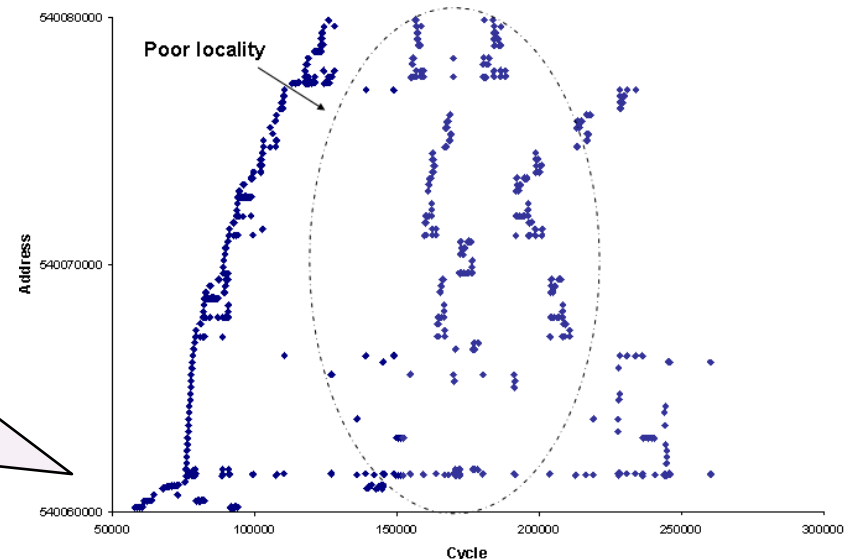


Good locality example:

The X axis corresponds to time (in cycles) and the Y axis corresponds to memory locations. The highlighted sections show examples of temporal and spatial locality.

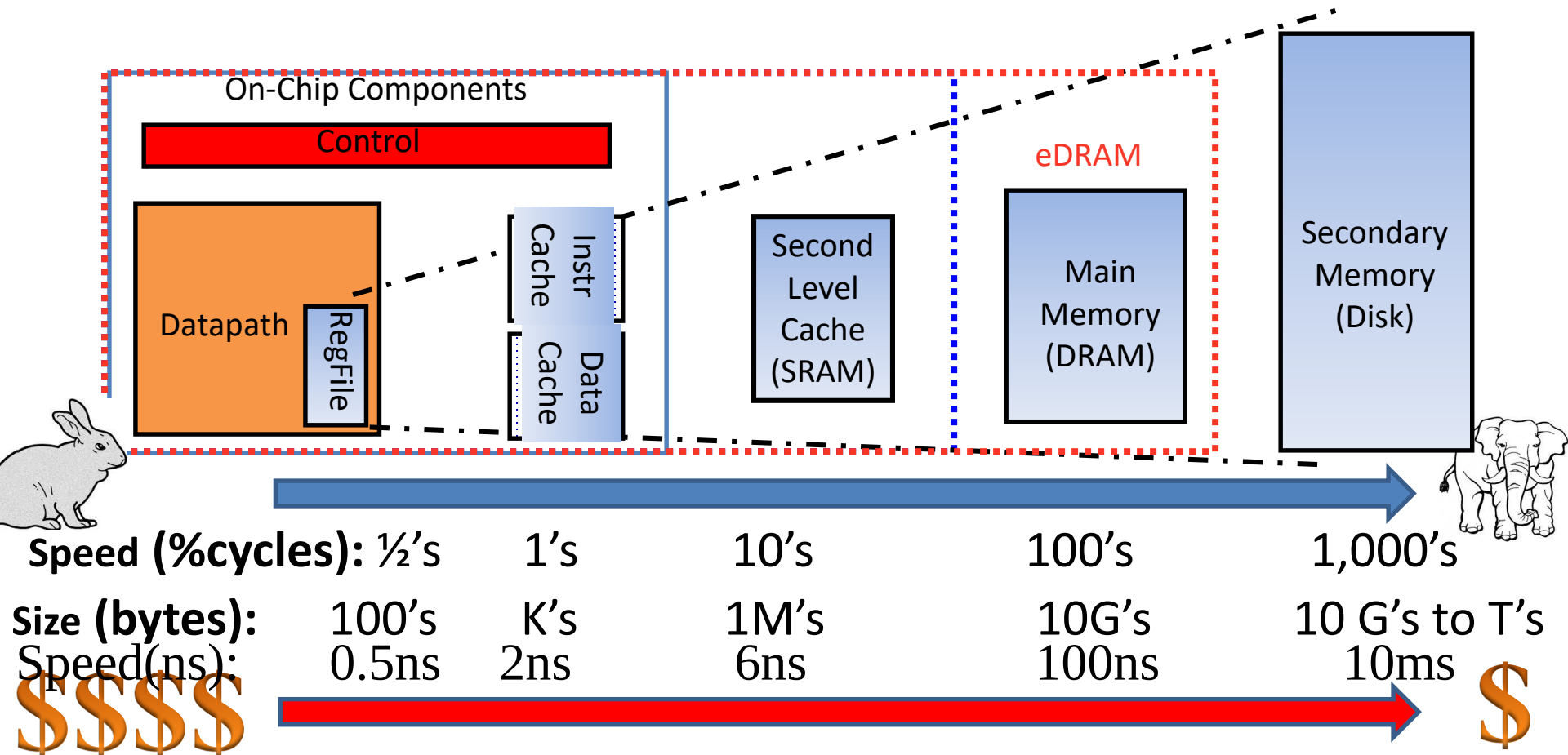
Poor locality example:

We can see that the program accesses a large memory footprint without a specific order, giving the pattern poor spatial locality. We also see little temporal re-use, which also makes this code cache unfriendly.



The Memory Hierarchy

- By taking advantage of the **principle of locality**
 - Can present the user with as much memory as is available in the cheapest technology
 - at the speed offered by the fastest technology



Cache Design

Maurice Wilkes, “Slave Memories and Dynamic Storage Allocation,” IEEE Trans. On Electronic Computers, 1965

Awards: Turing Award, IEEE John von Neumann Medal, Faraday Medal, Eckert–Mauchly Award, Mountbatten Medal



How it Really Looks Like

Desktop PC

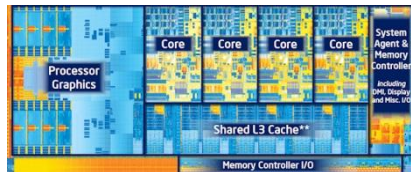


Source: Dell

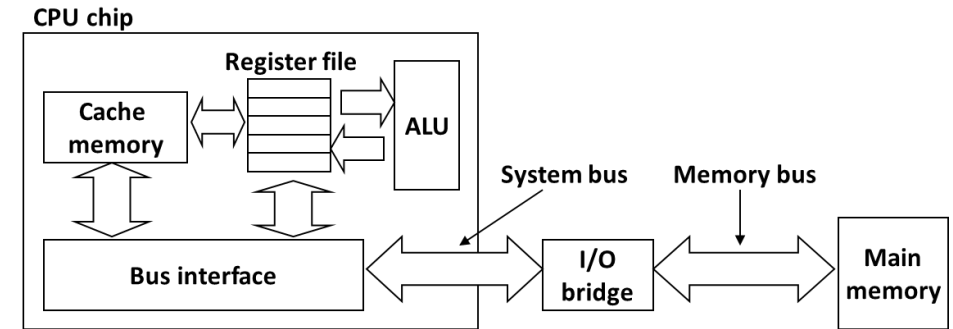
CPU (Intel Core i7)



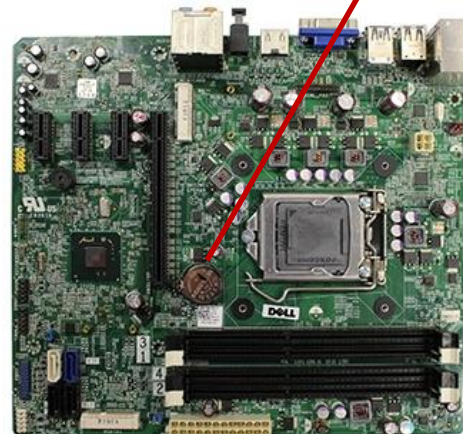
Source: PC Magazine



Source: techreport.com



Motherboard



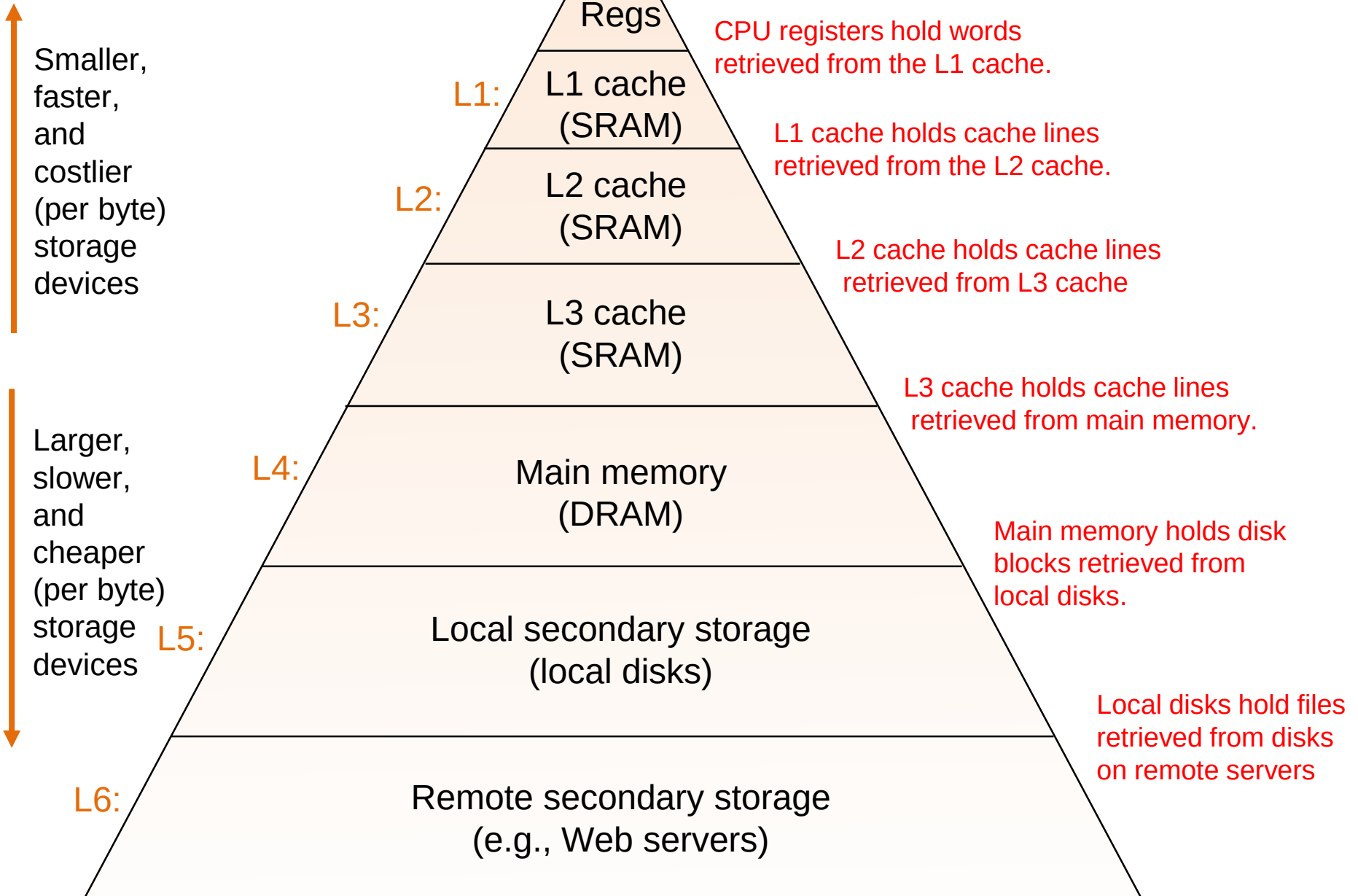
Source: Dell

Main memory (DRAM)

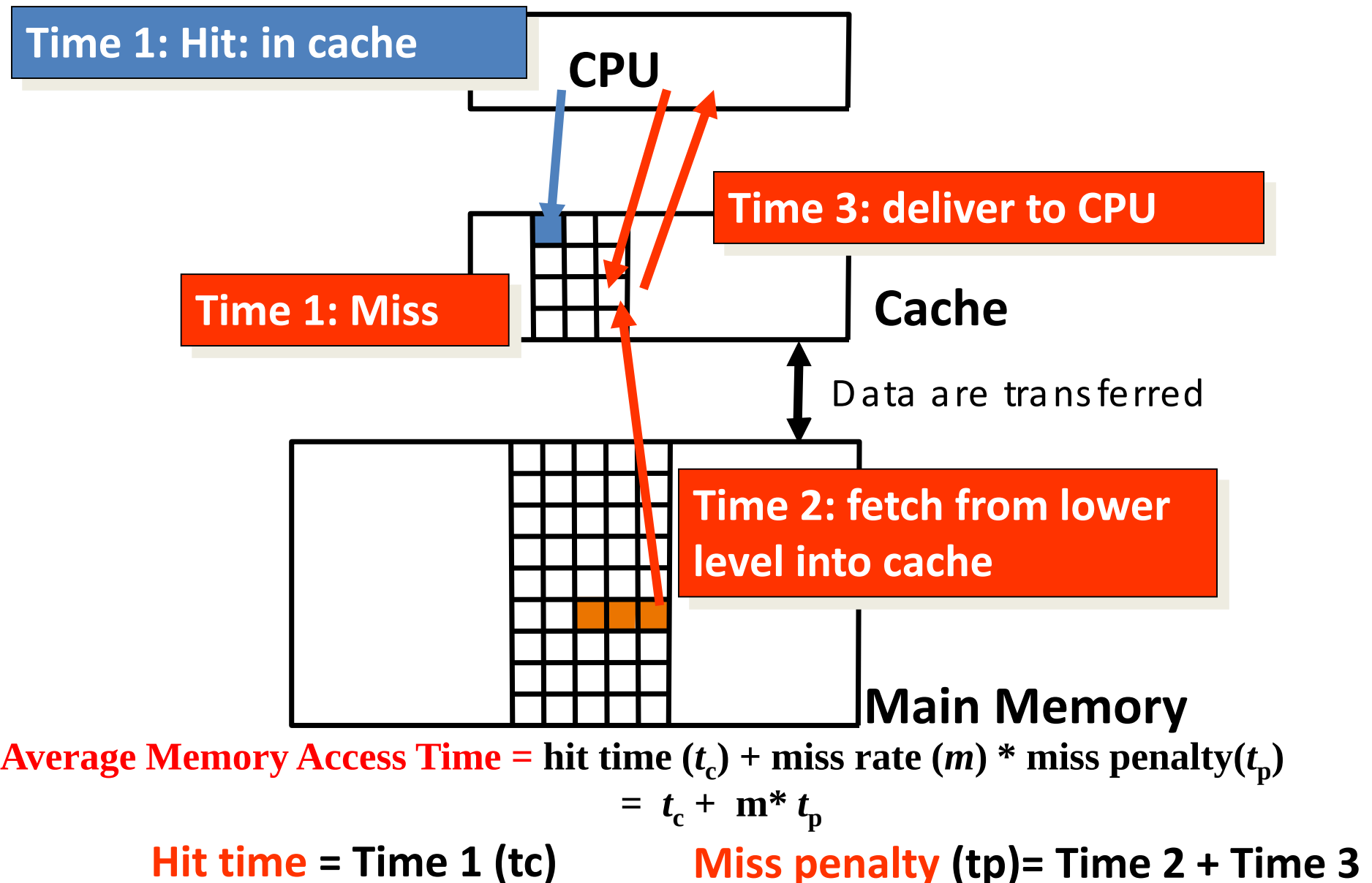


Source: Dell

Example Memory Hierarchy



Cache Example



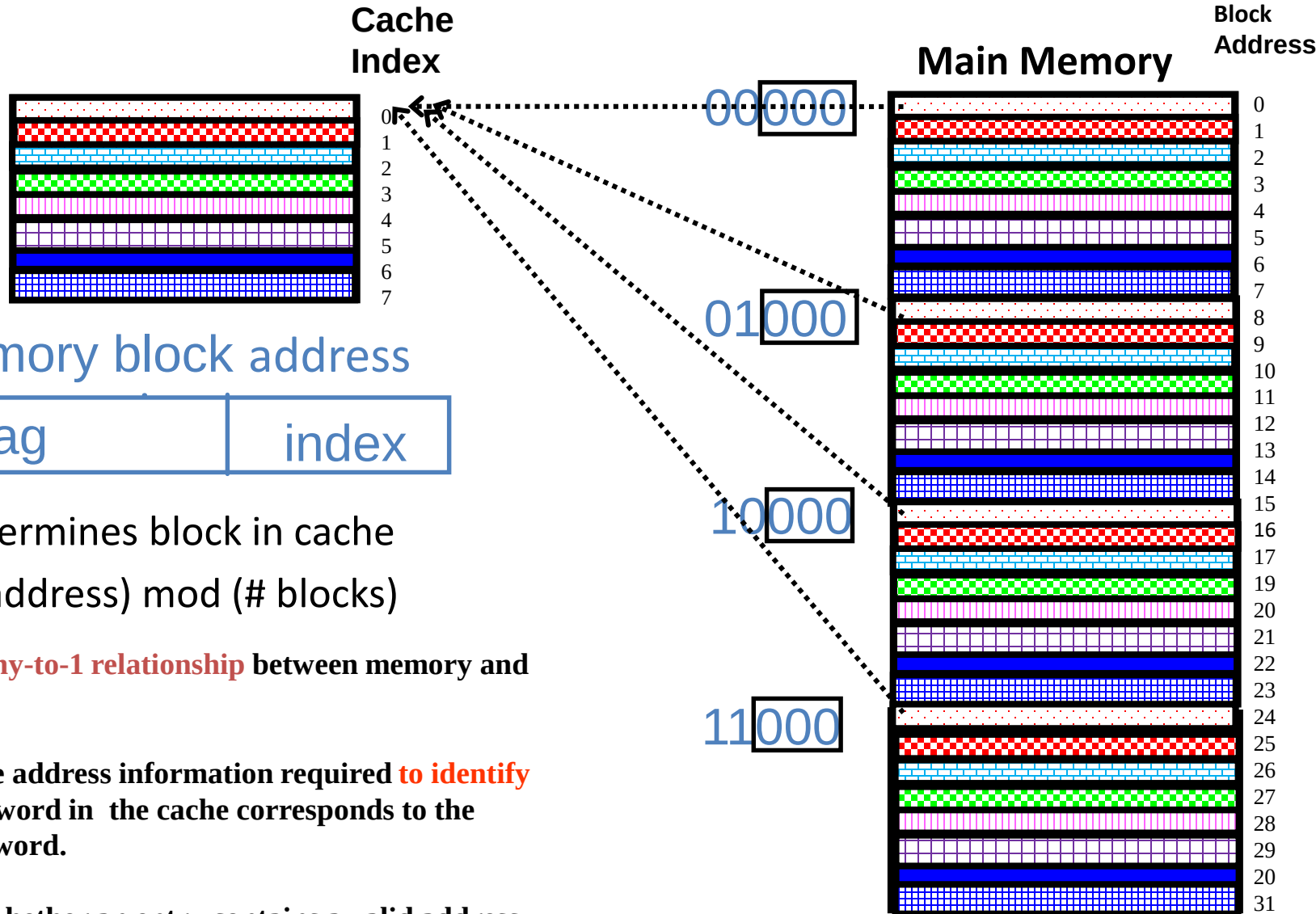
Example: Impact on Performance

- Suppose a processor executes at
 - Clock Rate = 1000 MHz (1 ns per cycle)
 - CPI = 1.1
 - 50% arith/logic, 30% ld/st, 20% control
- Suppose that 10% of memory operations get 50 cycle miss penalty
- $$\begin{aligned}\text{CPI} &= \text{ideal CPI} + \text{average stalls per instruction} \\ &= 1.1(\text{cyc}) + (0.30 (\text{datamops/ins}) \\ &\quad \times 0.10 (\text{miss/datamop}) \times 50 (\text{cycle/miss})) \\ &= 1.1 \text{ cycle} + \mathbf{0.3 \times 0.1 \times 50} \text{ cycle} = 1.1 \text{ cycle} + 1.5 \text{ cycle} \\ &= 2.6\end{aligned}$$
- 58 % of the time the processor is stalled waiting for memory!
- a 1% instruction miss rate would add an additional 0.5 cycles to the CPI!

Let's think about those numbers

- Huge difference between a hit and a miss
 - Could be 100x, if just L1 and main memory
- Would you believe 99% hits is twice as good as 97%?
 - Consider:
cache hit time of 1 cycle
miss penalty of 100 cycles
 - Average access time:
97% hits: $1 \text{ cycle} + 0.03 * 100 \text{ cycles} = 4 \text{ cycles}$
99% hits: $1 \text{ cycle} + 0.01 * 100 \text{ cycles} = 2 \text{ cycles}$
- This is why “miss rate” is used instead of “hit rate”

Simplest Cache: Direct Mapped



- index determines block in cache
- $\text{index} = (\text{address}) \bmod (\# \text{ blocks})$

There is a **Many-to-1 relationship** between memory and cache

tags

- contain the address information required to **identify** whether a word in the cache corresponds to the requested word.

valid bit

- indicates whether an entry contains a valid address

**Cache Table
Entry**

V	tag	Data
---	-----	------

Direct Mapped Cache: Temporal Example

lw \$1,10 110 (\$0)

lw \$2,11 010 (\$0)

lw \$3,10 110 (\$0)

Miss: valid

Miss: valid

Hit!

lw \$1,22(\$0)

lw \$2,26(\$0)

lw \$3,22(\$0)

Index	Valid	Tag	Data
000	N		
001	N		
010	Y	11	Memory[11010]
011	N		
100	N		
101	N		
110	Y	10	Memory[10110]
111	N		

Direct Mapped Cache: Worst case, always miss!

lw \$1,10 110 (\$0)

lw \$2,11 110 (\$0)

lw \$3,00 110 (\$0)

Miss: valid

Miss: tag

Miss: tag

lw \$1,22(\$0)

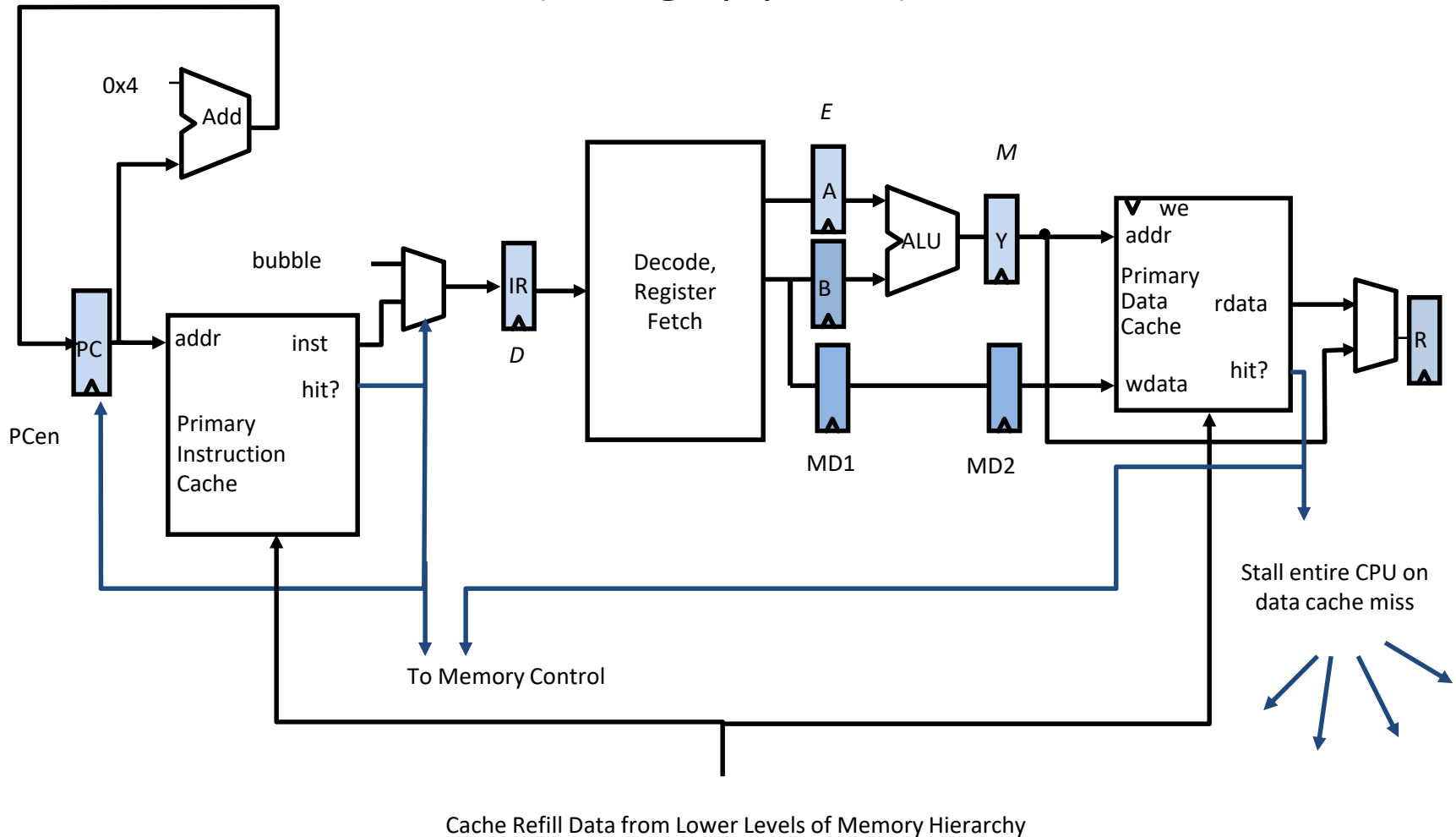
lw \$2,30(\$0)

lw \$3,6(\$0)

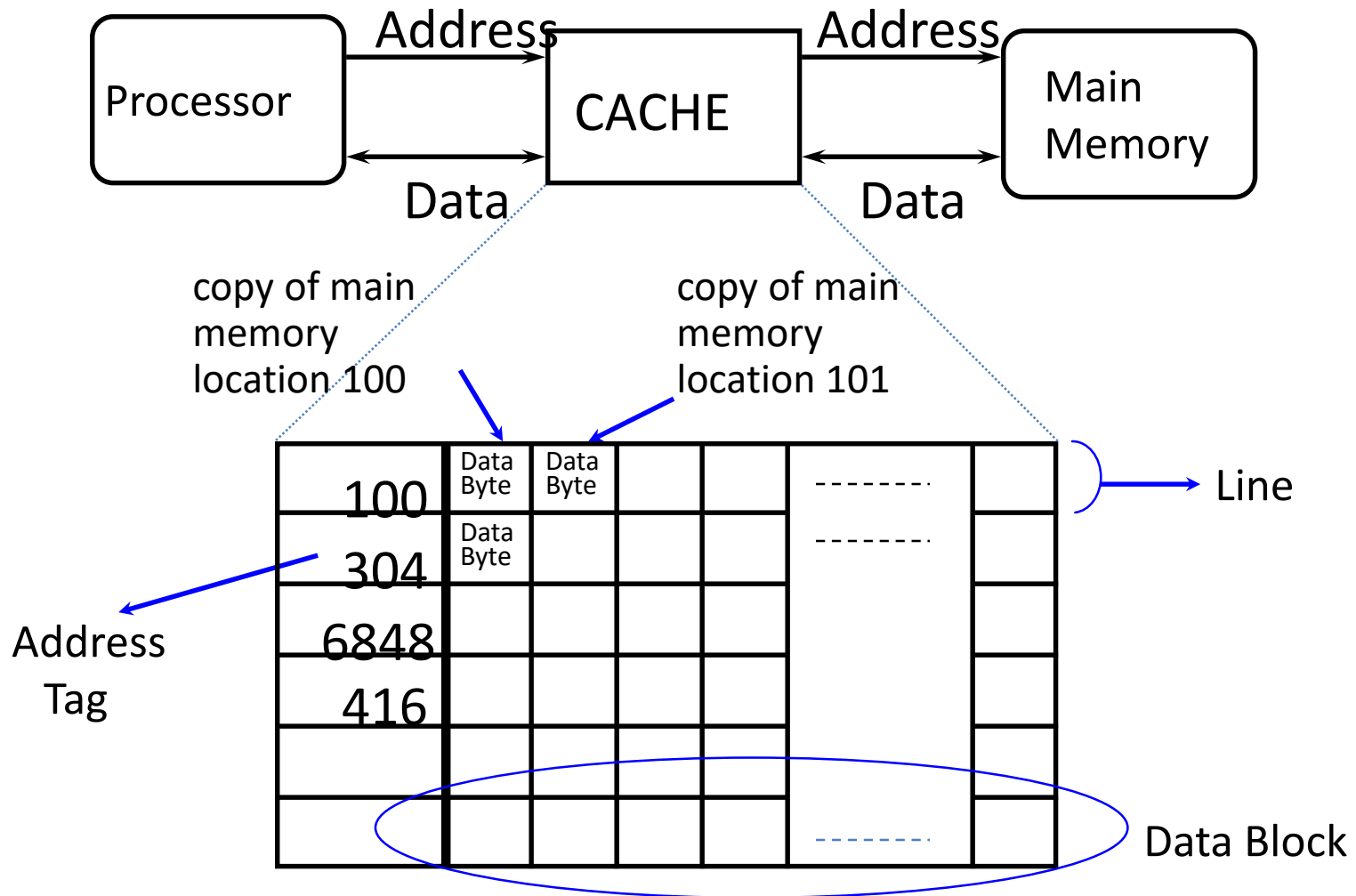
Index	Valid	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	00	Memory[00110]
111	N		

CPU-Cache Interaction

(5-stage pipeline)

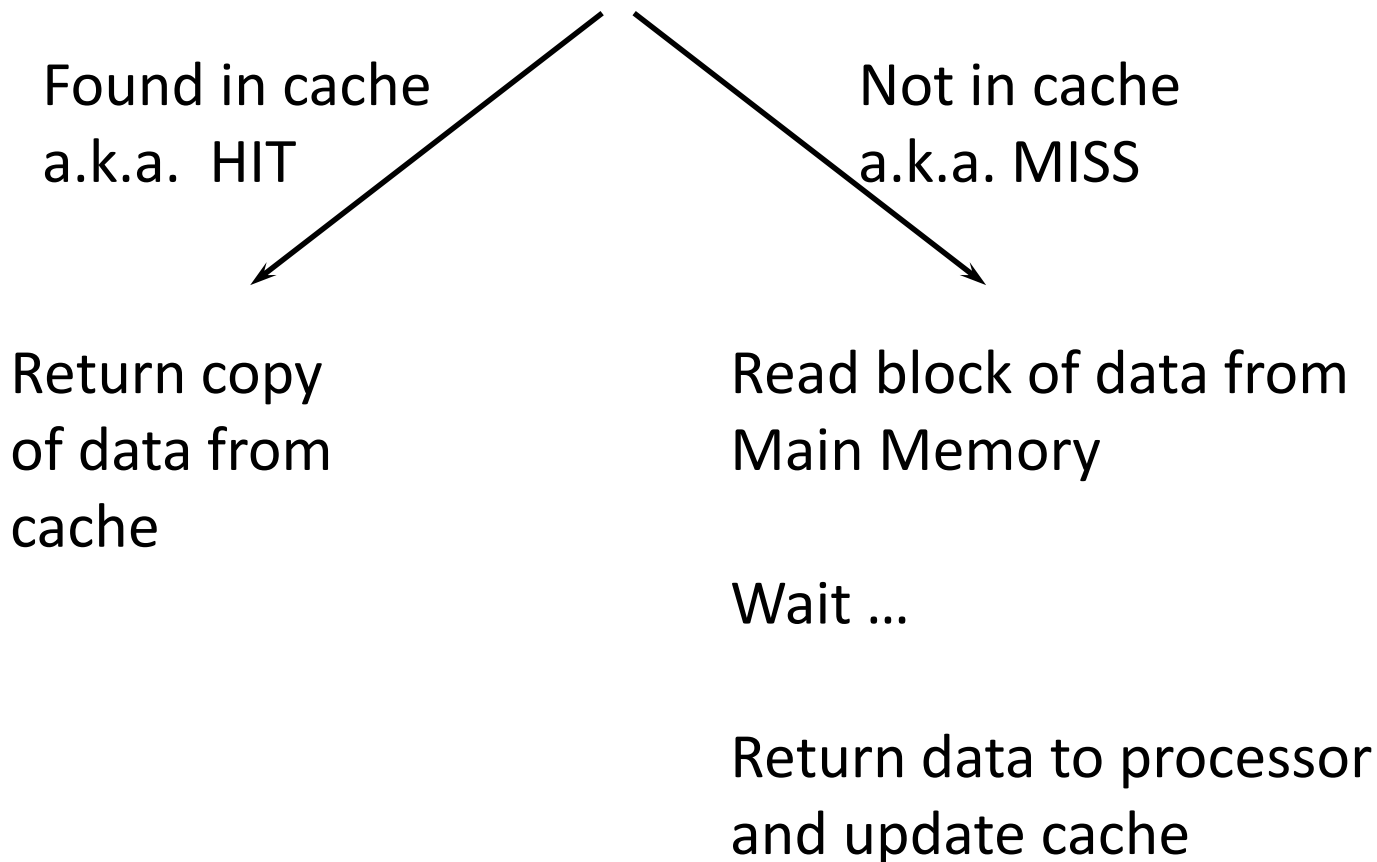


Inside a Cache



Cache Algorithm (Read)

Look at Processor Address, search cache tags to find match. Then either

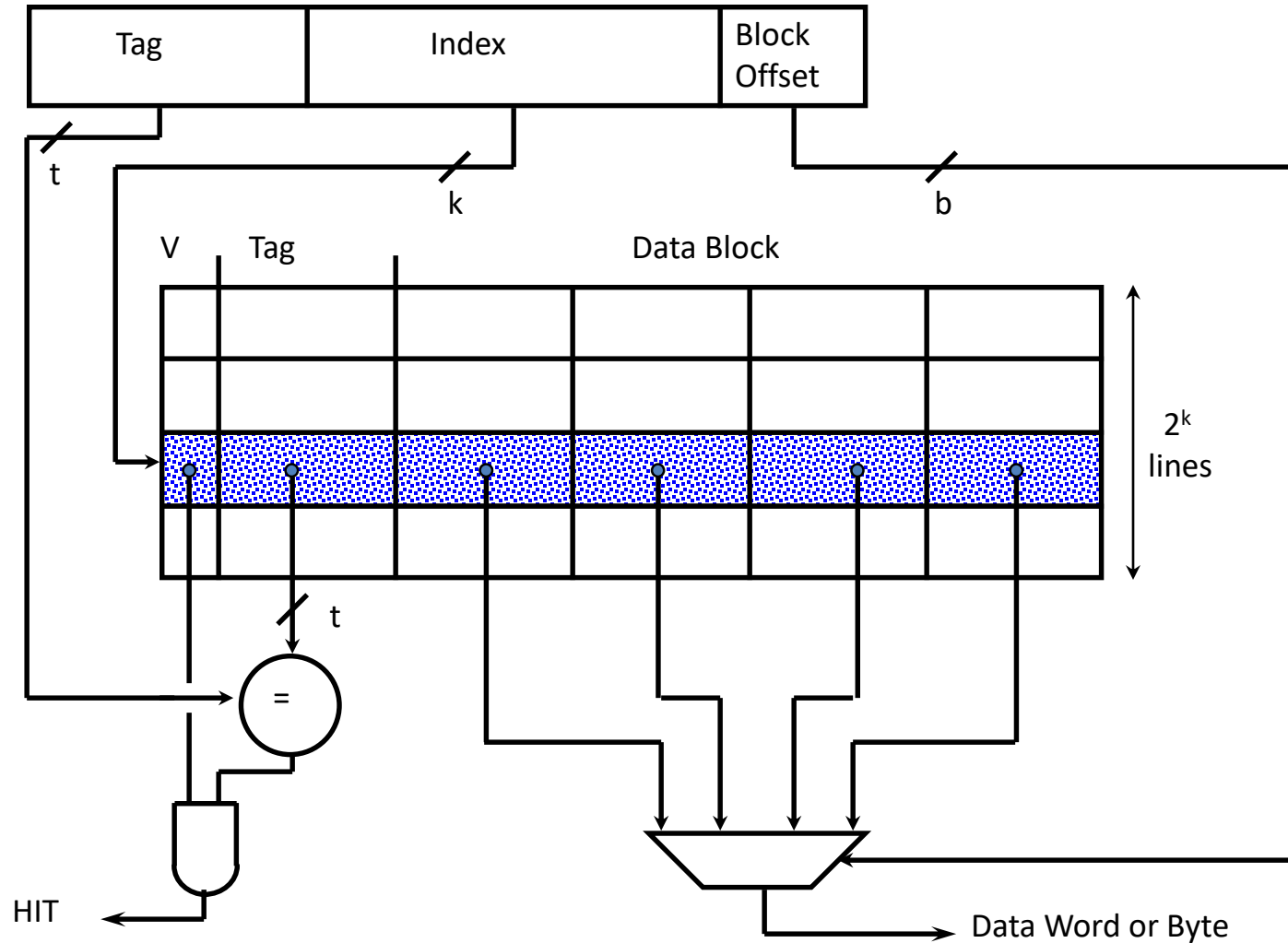


Q: Which line do we replace?

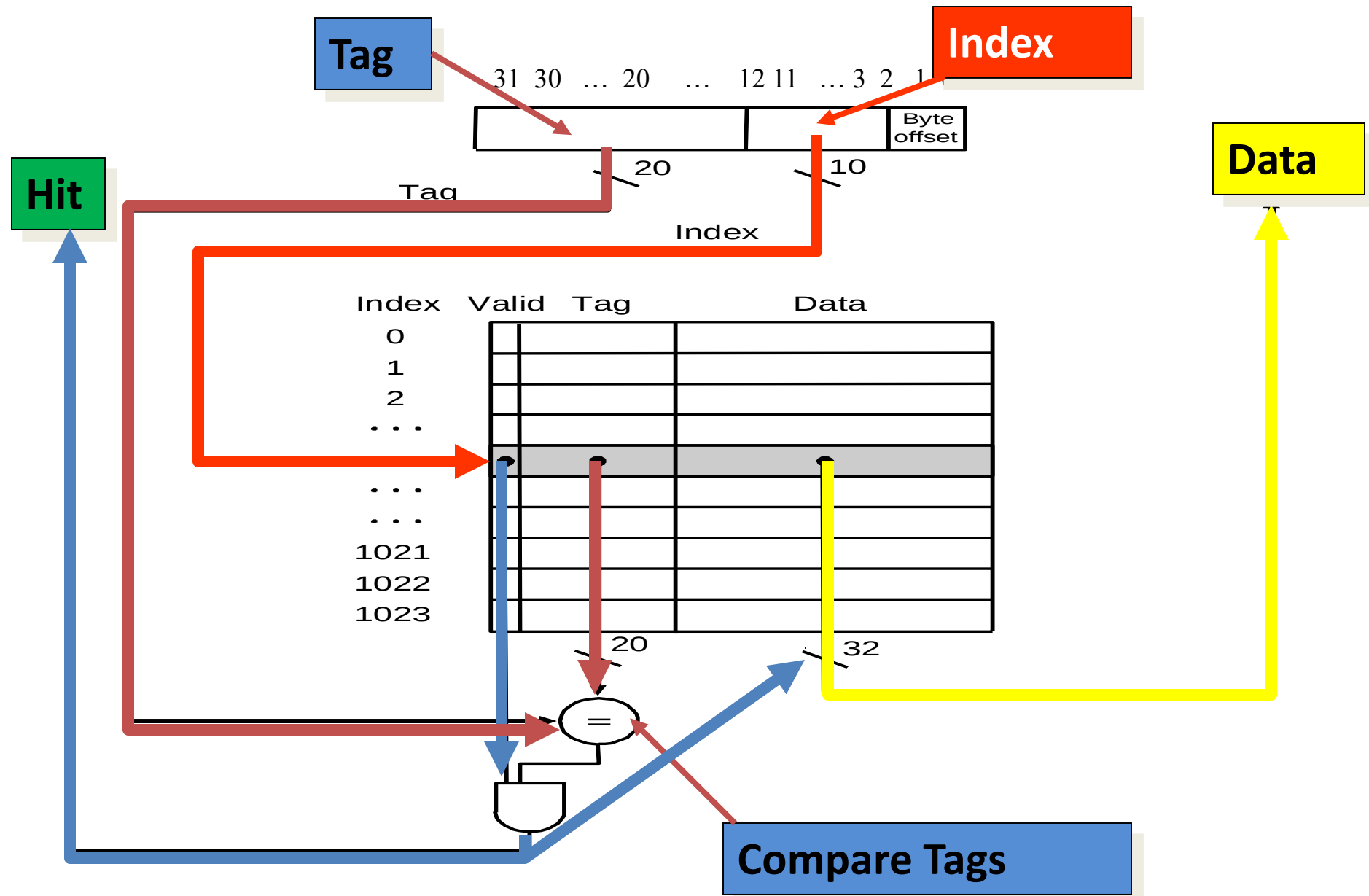
Caching Terminology

- **block/line** : the unit of information that can be exchanged between two adjacent levels in the memory hierarchy
- **cache hit**: when CPU finds a requested data item in the cache
- **cache miss**: when CPU does not find a requested data item in the cache
- **miss rate (m)** : fraction of cache accesses that result in a miss
- **hit rate (h)** : $(1 - \text{miss rate})$ fraction of memory access found in the faster level (cache)
- **hit time (t_c)** : time required to access the level 1 (cache)
- **miss penalty (t_p)** : time required to replace a data unit in the faster level (cache) + time involved to deliver the requested data to the processor from the cache. (hit time $\ll$ miss penalty)
- **average memory access time (amat)** = hit time (t_c) + miss rate (m) * miss penalty (t_p)
$$t_p = t_c + (1-h) * t_p$$

Direct-Mapped Cache

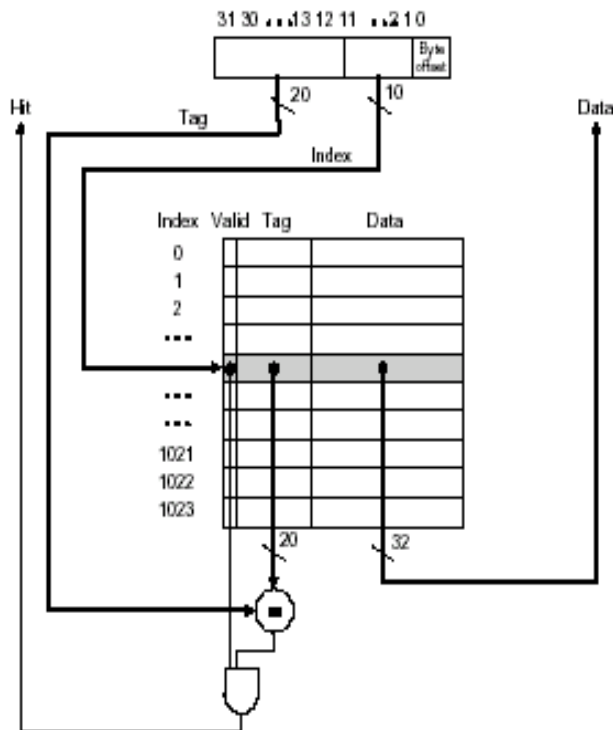


Direct Mapped Cache (4K): MIPS Example



Direct Mapped Cache [example]

- What is the size of cache ? 4K



- If I read
0000 0000 0000 0000 0000 0000 1000 0001

- What is the index number checked ?

0000 0000 0000 0000 0000 0000 0000 1000 0001

- If the number was found, what are the inputs to comparator ?

Direct Mapped Cache [contd...]

- Advantage
 - Simple
 - Fast
- Disadvantage
 - Mapping is fixed !!!

A computer system uses a 64 KB direct-mapped cache to speed up memory access. The cache is organized with 16-byte blocks, and the processor uses 32-bit memory addresses. Every time the CPU requests data, the cache controller must decide whether the required block is already present in the cache or whether it must be fetched from main memory. Design the cache by determining how many cache lines it contains and how a 32-bit memory address is divided into tag, index, and block offset fields

Nominal
Capacity

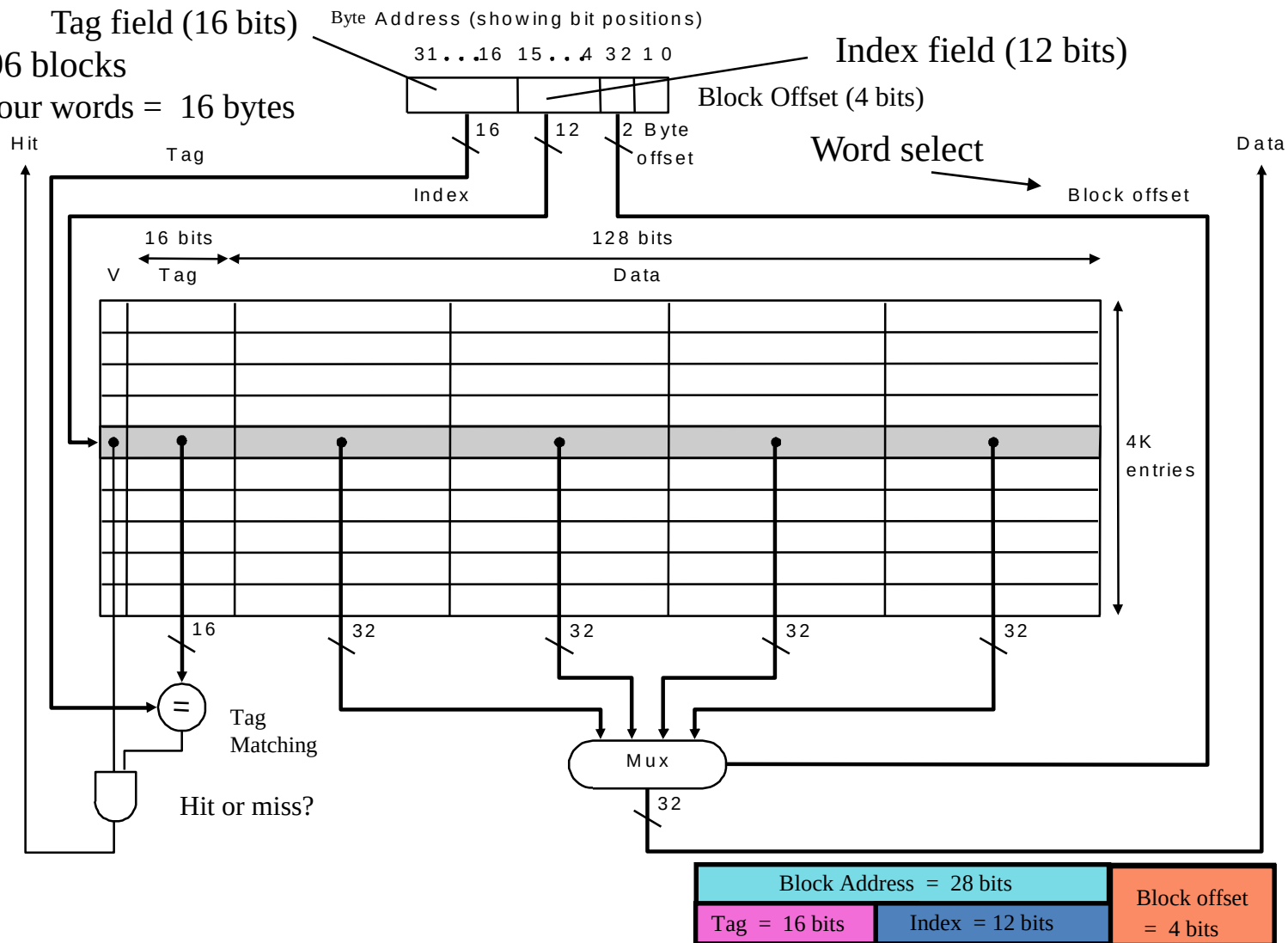
64KB Direct Mapped Cache Example

$$4K = 2^{12} = 4096 \text{ blocks}$$

Each block = four words = 16 bytes

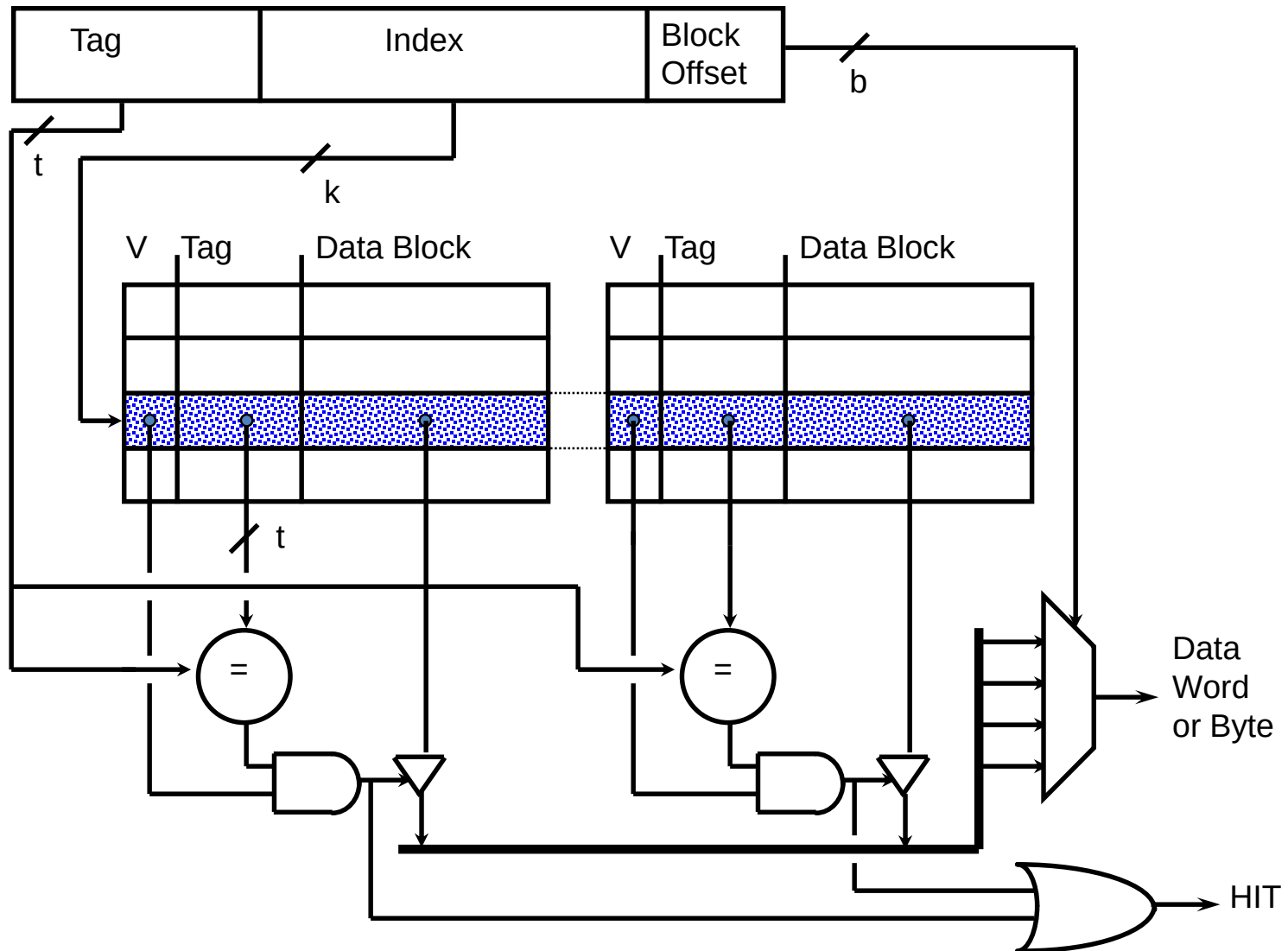
Can cache up to
 2^{32} bytes = 4 GB
of memory

SRAM

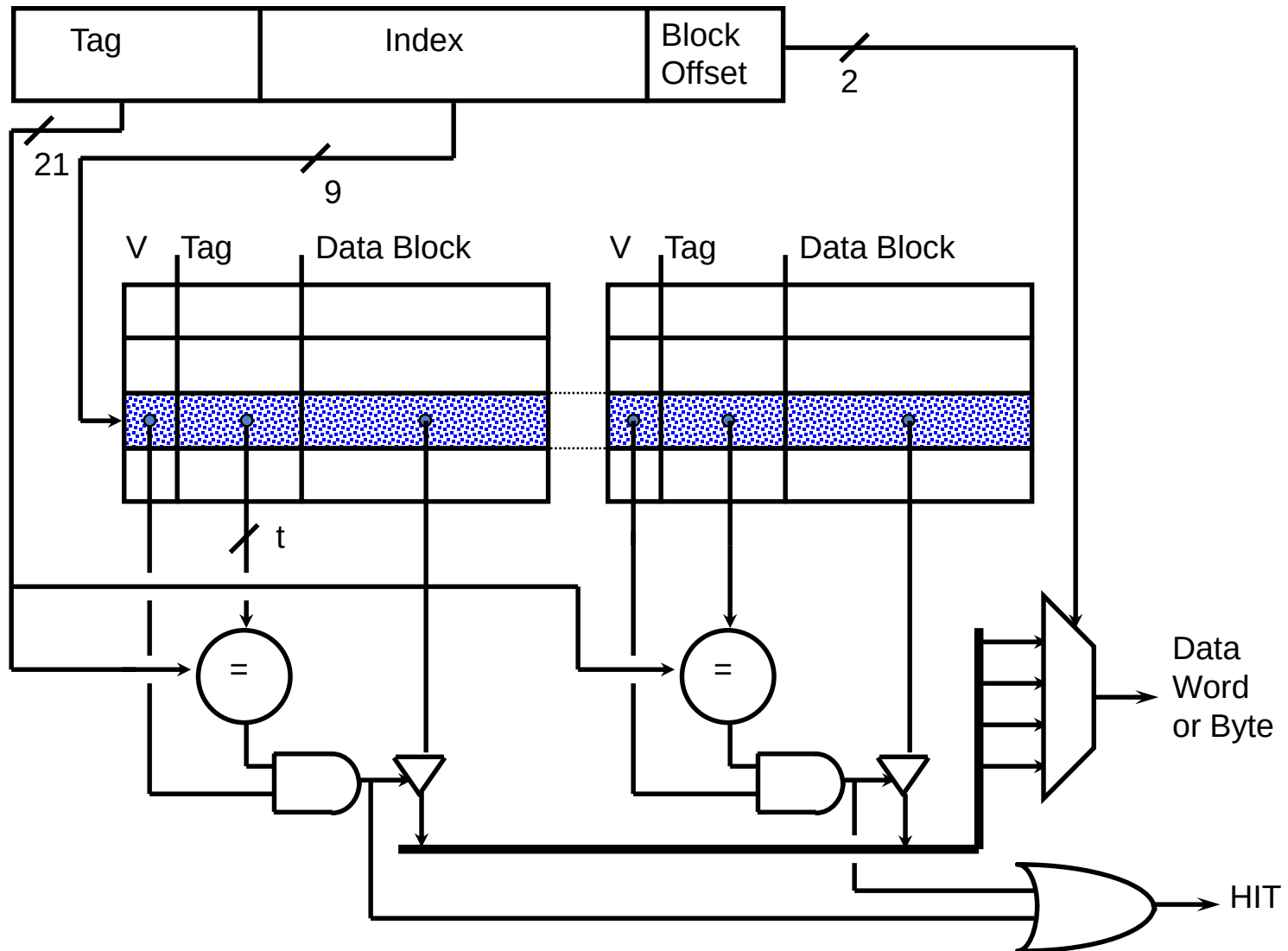


Mapping Function: Cache Block frame number = (Block address) MOD (4096)
i.e. index field or 12 low bit of block address

2-Way Set-Associative Cache



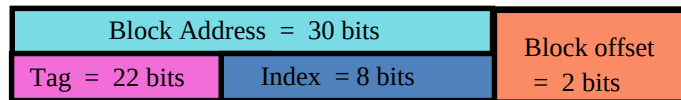
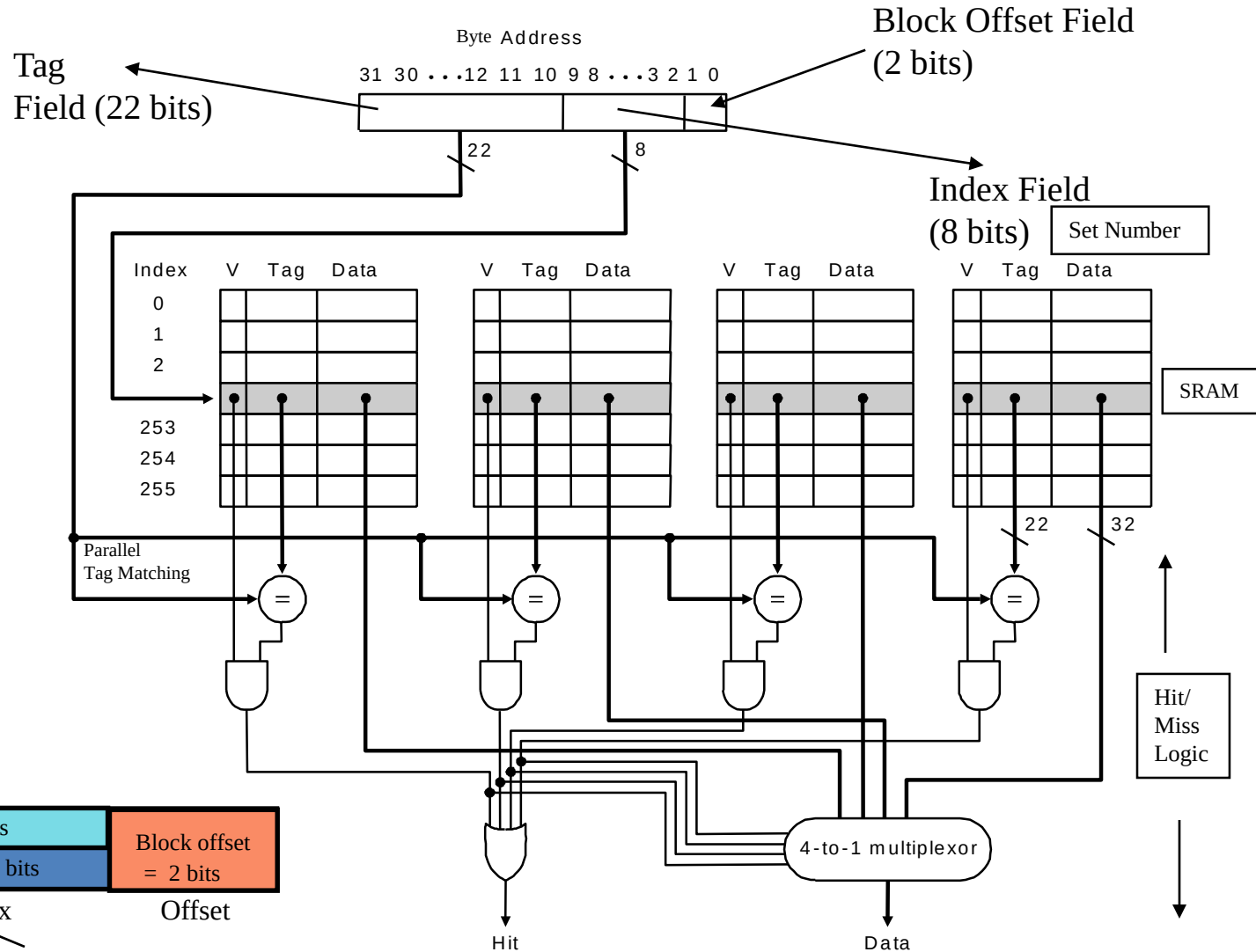
2-Way Set-Associative Cache(4K example)



4K Four-Way Set Associative Cache

4-way set associative
 $1024 / 4 = 2^8 = 256$ sets

Can cache up to
 2^{32} bytes = 4 GB



Tag

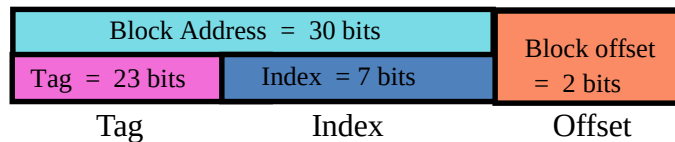
Index

Offset

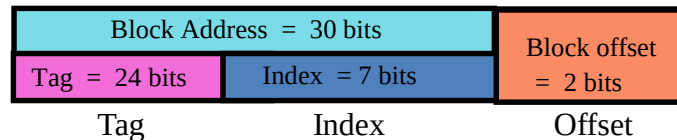
Mapping Function: Cache Set Number = index = (Block address) MOD (256)

4K (8, 16-Way ,.... Set Associative Cache

$$8 \text{ way} = 4\text{GB} = 32 \text{ bit} = 23(\text{tag}) + 7(\text{index}) + 2(\text{offset})$$



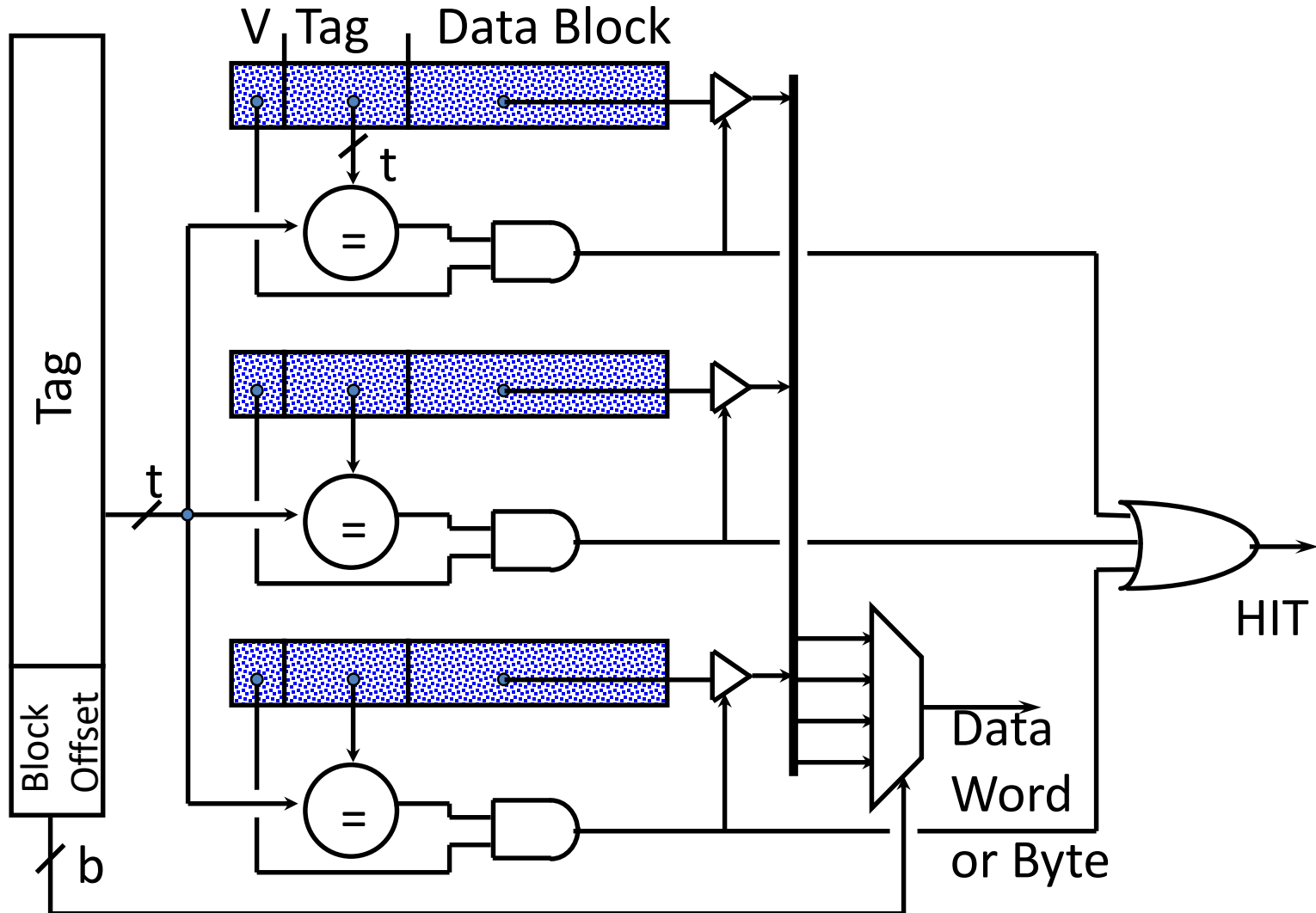
$$16 \text{ way} = 24(\text{tag}) + 6(\text{index}) + 2(\text{offset})$$



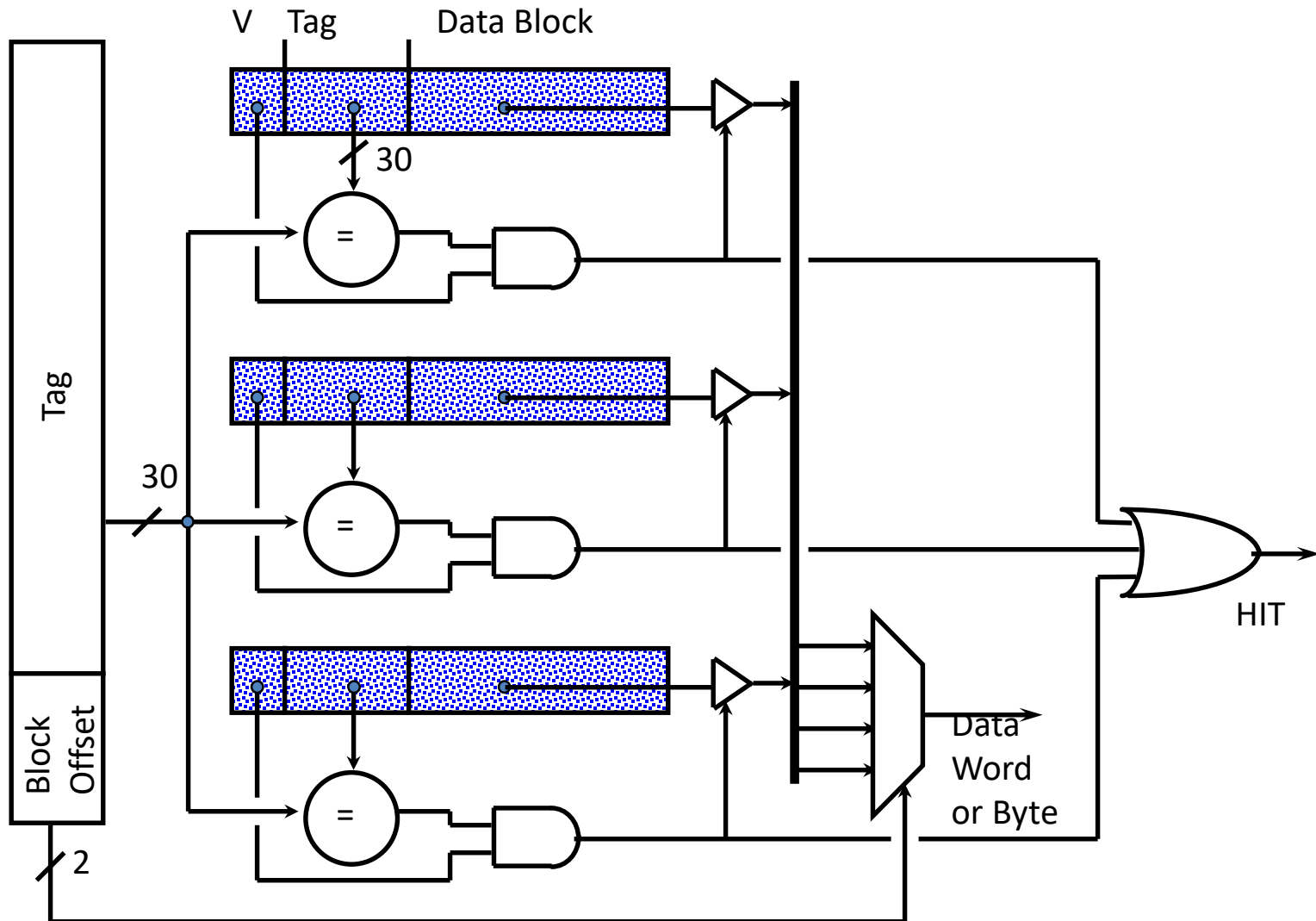
$$\text{Fully Associative} = 30(\text{tag}) + 2(\text{offset})$$



Fully Associative Cache



Fully Associative Cache(example)



Problem

- A computer system uses a fully associative cache to improve memory performance. The processor has a 32-bit address space, and the cache stores data in 32-byte blocks. Design the cache address format for this system by determining how many bits of the 32-bit address are used for the tag and how many are used for the block offset.

solution

- Fully Associative Cache

- No Cache Index
- Compare the Cache Tags of all cache entries in parallel
- 5 bits for block; 27 bits for Tag

Block Size = 32 B blocks, we need N 27-bit comparators (4K Cache, N=128; for a 32 KB cache N=1024)

